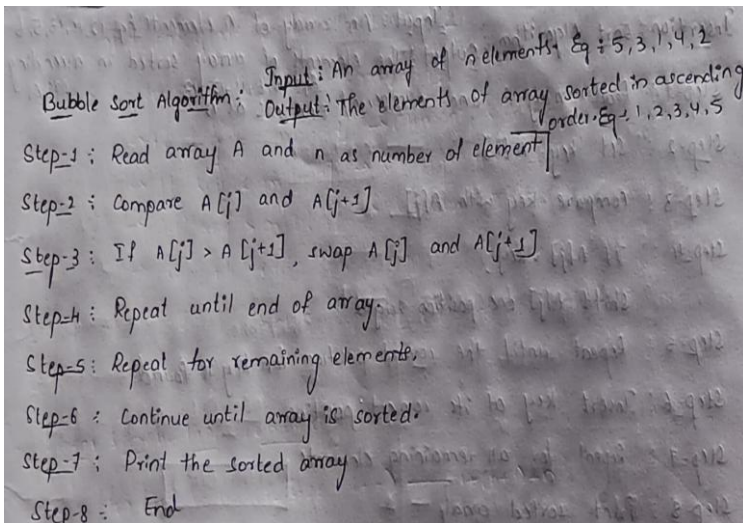


Practical - 1

Aim: To implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

1) Bubble Sort

Algorithm:



Program:

```
#include <iostream>
using namespace std;
```

```
class BubbleSort
{
public:
    int arr[100], n;

    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
}
```



```
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void logic()
{
    int temp;

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

};

int main()
{
    BubbleSort bs;

    bs.arrayInput();

    cout << "\nBefore Sorting: ";
    bs.printArray();

    bs.logic();

    cout << "After Sorting: ";
    bs.printArray();

    return 0;
}
```

Output:

```
enter program name, for example C:\Users\vemal\OneDrive\Documents\DSA/a.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ bubble.cpp -o bubble.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\bubble.exe
Enter number of elements: 5
Enter the elements: 19 22 12 16 30

Before Sorting: 19 22 12 16 30
After Sorting: 12 16 19 22 30
PS C:\Users\vemal\OneDrive\Documents\DSA> |
```

Time Complexity:

Bubble Sort Time Complexity:

for (int i = 0; i < n-1; i++) $\rightarrow O(n) - n-1$

{ for (j = 0; j < n-i-1; j++) $\rightarrow O(n) - n-1$ (The inner loop runs:

{ if (A[j] > A[j+1]) $\rightarrow O(1)$ $i=0 \rightarrow n-1$ comparisons

{ temp = A[j]; $\rightarrow O(1)$ $i=1 \rightarrow n-2$ comparisons

A[j] = A[j+1]; $\rightarrow O(1)$ $i=2 \rightarrow n-3$ comparisons

A[j+1] = temp; $\rightarrow O(1)$ $i=n-2 \rightarrow 1$ comparison

}

}

Therefore,

$$T(n) = (n-1) + (n-2) + (n-3) + \dots + 1$$

$$= \frac{n(n-1)}{2} = \frac{n^2-n}{2} = O(n^2)$$

$\therefore T(n) = O(n^2)$

Best case: If the array is already sorted.

Ex: 1 2 3 4 5 (No swapping is required)

$T(n) = O(n)$

Average case: For a random arrangement, comparisons are still:

$(n-1) + (n-2) + \dots + 1$ (Some elements require swapping & some don't)

$T(n) = O(n^2)$

Worst case: If the array is reverse order. Ex: 5 4 3 2 1

$T(n) = n(n-1) = O(n^2)$

Bubble sort Conclusion :

Thus, the Bubble Sort algorithm was successfully implemented and studied. It sorts the elements by repeatedly comparing adjacent elements and swapping them when they are in the wrong order. It is simple and easy to understand, but it is less efficient for large datasets because its average and worst-case time complexity is $O(n^2)$.

2) Selection sort:

Algorithm:

Input : An array of n elements. Eg: 64, 25, 12, 22, 11
Output : The elements of array sorted in ascending order. Eg: 11, 12, 22, 25, 64

Selection sort Algorithm:

Step-1 : Read array A and number of elements n

Step-2 : set $min = i$

Step-3 : compare $A[min]$ with $A[j]$

Step-4 : If $A[j] < A[min]$
set $min = j$

Step-5 : Repeat step 3 and step 4 until the end of array

Step-6 : swap $A[i]$ and $A[min]$

Step-7 : Repeat the above steps for remaining elements

Step-8 : Print sorted array.

Step-9 : End

Program :

```
#include <iostream>
using namespace std;

class SelectionSort
{
public:
    int arr[100], n;

    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }

    void logic()
    {
        int i, j, min, temp;

        for (i = 0; i < n - 1; i++)
        {
            min = i;
```



```
        for (j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[min])
            {
                min = j;
            }
        }

        swap(arr[i], arr[min]);
    }
}

void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

    cout << endl;
}

};

int main()
{
    SelectionSort ss;

    ss.arrayInput();

    cout << "Before Sorting: ";
    ss.printArray();

    ss.logic();

    cout << "After Sorting: ";
    ss.printArray();

    return 0;
}
```

Output:

```
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ selection.cpp -o selection.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\selection.exe
Enter number of elements: 5
Enter the elements: 22 30 12 16 19
Before Sorting: 22 30 12 16 19
After Sorting: 12 16 19 22 30
```

Time complexity:

Selection Sort Time Complexity:

```

int minIndex, temp;
for (int i = 0; i < n-1; i++)
{
    minIndex = i;
    for (int j = i+1; j < n; j++)
    {
        if (A[j] < A[minIndex])
        {
            minIndex = j;
        }
    }
    temp = A[i];
    A[i] = A[minIndex];
    A[minIndex] = temp;
}

```

The inner loop performs:

$$(n-1) + (n-2) + (n-3) + \dots + 1$$

$$= \frac{n(n-1)}{2} \approx O(n^2)$$

Best case: The array is already sorted
 Eg: 10 20 30 40 50 (Even though the array is sorted, selection sort still checks every remaining element to find the minimum)

$$\Rightarrow (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$$
 Therefore, Best case = $O(n^2)$

Average case: Eg: 30 10 50 20 40 (The elements are in random order)
 For n elements, $\frac{n(n-1)}{2}$
 Average case = $O(n^2)$

Worst case: Eg: 50 40 30 20 10 (The array is in reverse order)
 The no. of comparisons remains: $\frac{n(n-1)}{2}$
 Worst case = $O(n^2)$

Selection sort Conclusion :

Thus, the Selection Sort algorithm was successfully implemented and studied. It sorts the elements by repeatedly finding the smallest element from the unsorted part and placing it in the correct position. It is simple and easy to understand, but its time complexity is $O(n^2)$ in the best, average, and worst cases.

3) Insertion sort:

Algorithm:

Insertion sort Algorithm: Input: An array of n elements. Output: The elements of array sorted in ascending order.

Step-1: Read array A and number of elements n .

Step-2: Set $key = A[i]$.

Step-3: Compare key with $A[j]$.

Step-4: If $A[j] > key$, shift $A[j]$ one position right.

Step-5: Repeat until the correct position for key is found.

Step-6: Insert key at its correct position.

Step-7: Repeat for all remaining elements.

Step-8: Print sorted array.

Step-9: End.

Program:

```
#include <iostream>
using namespace std;

class InsertionSort
{
public:
    int arr[100], n;

    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }

    void logic()
    {
        int i, j, key;
```



```
        for (i = 1; i < n; i++)
        {
            key = arr[i];

            while (j >= 0 && arr[j] > key)
            {
                arr[j + 1] = arr[j];
                j--;
            }

            arr[j + 1] = key;
        }
    }

    void printArray()
    {
        for (int i = 0; i < n; i++)
        {
            cout << arr[i] << " ";
        }

        cout << endl;
    }
};

int main()
{
    InsertionSort is;

    is.arrayInput();

    cout << "Before Sorting: ";
    is.printArray();

    is.logic();

    cout << "After Sorting: ";
    is.printArray();

    return 0;
}
```

Output:

```
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ insertion.cpp -o insertion.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\insertion.exe
Enter number of elements: 5
Enter the elements: 22 12 16 19 30
Before Sorting: 22 12 16 19 30
After Sorting: 12 16 19 22 30
```

Time Complexity :

Insertion sort Time complexity :-

```

for (i=0; i<n-1; i++)      ——— n-1
{
    int min=i;              ——— 1
    for (j=i+1; j<n; j++)    ——— n-i
    {
        if (arr[j]<arr[min]) ——— 1
        {
            min=j;          ——— 1
        }
    }
    if (min!=i)              ——— 1
    {
        int temp=arr[i];     ——— 1
        arr[i] = arr[min];   ——— 1
        arr[min] = temp;     ——— 1
    }
}

```

$T = (n-1)(n-1) + (1+1+1+1+1)$
 $= 5n + 2n$
 $T = O(n^2)$

Best case: Eg: 10 20 30 40 50 (the array is sorted)
 → only one comparison is needed for each element, and no shifting is required.
Best case = $O(n)$

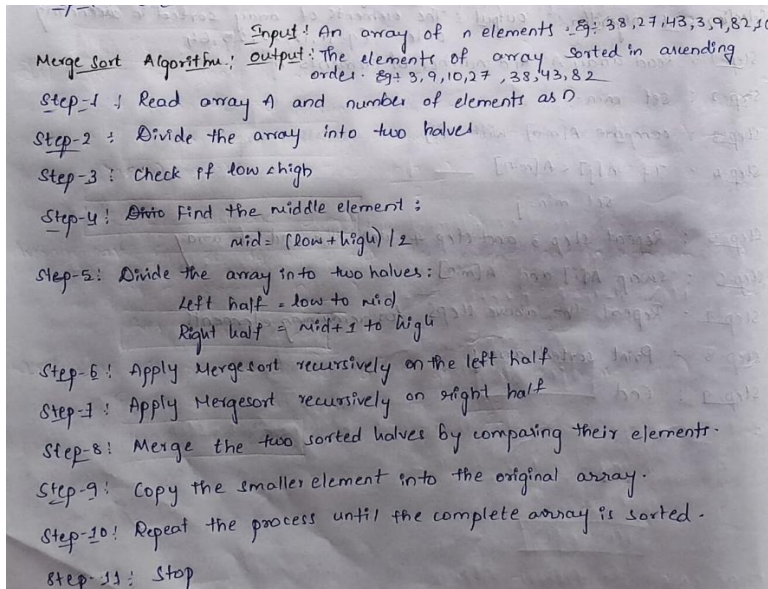
Average case: Eg: 30 50 20 40 30
 Elements need to be compared and shifted to their correct positions.
Average case = $O(n^2)$

Worst case: Eg: 50 40 30 20 10
 Every element has to be compared with and shifted past almost all previous elements.
Worst case = $O(n^2)$

Insertion sort Conclusion :

Thus, the Insertion Sort algorithm was successfully implemented and studied. It sorts the elements by taking one element at a time and inserting it into its correct position in the sorted part of the array. It is simple and efficient for small or nearly sorted data. Its best-case time complexity is **$O(n)$** , while its average and worst-case time complexity is **$O(n^2)$** .

4) Merge Sort Algorithm :



Program :

```
#include <iostream>
using namespace std;
```

```
class MergeSort
{
public:
    int arr[100], n;

    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
}
```



```
void merge(int low, int mid, int high)
{
    int temp[100];

    int i = low;
    int j = mid + 1;
    int k = low;

    while (i <= mid && j <= high)
    {
        if (arr[i] < arr[j])
        {
            temp[k] = arr[i];
            i++;
        }
        else
        {
            temp[k] = arr[j];
            j++;
        }

        k++;
    }

    while (i <= mid)
    {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= high)
    {
        temp[k] = arr[j];
        j++;
        k++;
    }

    for (i = low; i <= high; i++)
    {
        arr[i] = temp[i];
    }
}
```

```
void logic(int low, int high)
{
```



```
    if (low < high)
    {
        int mid = (low + high) / 2;

        logic(low, mid);
        logic(mid + 1, high);
        merge(low, mid, high);
    }
}

void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

    cout << endl;
}

};

int main()
{
    MergeSort ms;

    ms.arrayInput();

    cout << "Before Sorting: ";
    ms.printArray();

    ms.logic(0, ms.n - 1);

    cout << "After Sorting: ";
    ms.printArray();

    return 0;
}
```

Output:

```
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ merge.cpp -o merge.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\merge.exe
Enter number of elements: 5
Enter the elements: 10 20 40 22 19
Before Sorting: 10 20 40 22 19
After Sorting: 10 19 20 22 40
```

Time Complexity :

Merge Sort Time Complexity:-

mergesort(arr, low, mid);
mergesort(arr, mid+1, high);
merge(arr, low, mid, high);

$$T(n) = T(n/2) + T(n/2) + O(n)$$

$$T(n) = 2T(n/2) + O(n)$$

Master Method: $T(n) = aT(n/b) + f(n)$
 $a = 2, b = 2, f(n) = n, p = 0, k = 1$
 $n^{\log_b a} = n^{\log_2 2} = n$
 $f(n) = n$
 $n^{\log_2 2} = n \therefore \log_2 2 = 1$

case-2: $p > -1$
 $T(n) = O(n \log^{p+1} n)$
 $T(n) = O(n \log n)$

Best case! Eg: 1 2 3 4 5 6
 The array is sorted and merge sort still divides the array and merges the parts.
Best case = $O(n \log n)$

Average case! Eg: 4 2 6 1 5 3
 The array is divided into smaller parts and then merged in sorted order.
Average case = $O(n \log n)$

Worst case! Eg: 6 5 4 3 2 1
 The array is in reverse order and merge sort still divides and merges all elements.
Worst case = $O(n \log n)$

Merge sort Conclusion :

Thus, the Merge Sort algorithm was successfully implemented and studied. It sorts the elements by dividing the array into smaller parts and then merging them in sorted order. It is efficient for large datasets and has a time complexity of **$O(n \log n)$** in the best, average, and worst cases.

5) Quick Sort:

Algorithm:

Quicksort Algorithm:
Input: An array of n elements. Eg: 10, 7, 8, 9, 15
Output: The elements of array sorted in ascending order. Eg: 7, 8, 9, 10, 15
Algo: Step-1: Start
Step-2: Check whether $low < high$
Step-3: If $low < high$, select pivot element from the array
Step-4: Partition the array around the pivot such that:
- Elements smaller than or equal to the pivot are placed on the left.
- Elements greater than the pivot are placed on the right.
Step-5: After partitioning, the pivot is placed in its correct position. Let the position be p
Step-6: Recursively apply Quicksort to the left sub array;
 $QUICKSORT(A, low, p-1)$
Step-7: Recursively apply Quicksort to the right sub array
 $QUICKSORT(A, p+1, high)$
Step-8: Repeat the process until $low \geq high$
Step-9: Stop

Program :

```
#include <iostream>

using namespace std;

class QuickSort
{
public:
    int arr[100], n;

    void arrayInput()
    {
        cout << "Enter number of elements: ";

        cin >> n;
```



```
cout << "Enter the elements: ";

for (int i = 0; i < n; i++)

{

    cin >> arr[i];

}

}

void printArray()

{

for (int i = 0; i < n; i++)

{

cout << arr[i] << " ";

}

cout << endl;

}

int partition(int arr[], int lb, int ub)

{

    int pivot = arr[lb];

    int start = lb;

    int end = ub;

    while (start < end)

    {

while (arr[start] <= pivot && start < ub)

{

start++;

}

}
```



```
while (arr[end] > pivot)
{
    end--;
}

if (start < end)
{
    swap(arr[start], arr[end]);
}

swap(arr[lb], arr[end]);

return end;
}

void quicksort(int arr[], int lb, int ub)
{
    if (lb < ub)
    {
        int location = partition(arr, lb, ub);
        quicksort(arr, lb, location - 1);
        quicksort(arr, location + 1, ub);
    }
}

void sort()
{
    quicksort(arr, 0, n - 1);
}
```

};

int main()

{

QuickSort qs;

qs.arrayInput();

cout << "\nBefore Sorting: ";

qs.printArray();

qs.sort();

cout << "After Sorting: ";

qs.printArray();

return 0;

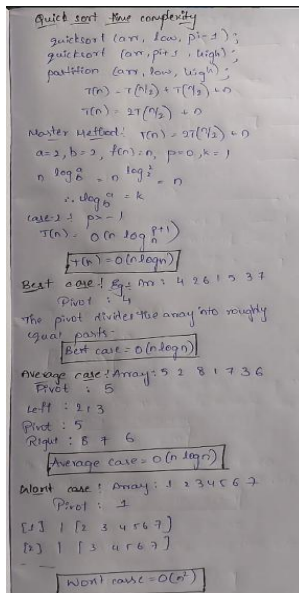
}

Output:

```
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ quick.cpp -o quick.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\quick.exe
Enter number of elements: 5
Enter the elements: 12 14 34 78 56

Before Sorting: 12 14 34 78 56
After Sorting: 12 14 34 56 78
```

Time complexity :



Quicksort Conclusion :

Thus, the Quick Sort algorithm was successfully implemented and studied. It sorts the elements by selecting a pivot and partitioning the array around the pivot. It is generally efficient for large datasets, with an average-case time complexity of $O(n \log n)$. Its best-case complexity is $O(n \log n)$, while the worst-case complexity is $O(n^2)$.

Conclusion:

Thus, the sorting algorithms Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort were successfully implemented and their time complexities were analysed. The practical helped in understanding their performance and efficiency for different input sizes.